

Algorithmes pilotés par les données

Correction orthographique, dictionnaires et apprentissage automatique

Traduit et adapté depuis Bootstrap World (Fall 2026)

Adapté au contexte francophone — accents, dictionnaires français, exemples en français

Licence Creative Commons 4.0 International

*Ces supports ont été développés en partie grâce au soutien de la National Science Foundation
(bourses 1042210, 1535276, 1648684, 1738598, 2031479 et 1501927).*

Table des matières

1	Correction orthographique : l'algorithme	3
1.1	Lancement	3
1.2	Exploration	4
1.3	Synthèse	5
2	Correction orthographique en Pyret	6
2.1	Lancement	6
2.1.1	Particularités du français	6
2.2	Exploration	7
2.3	Synthèse	9
3	Définir des valeurs	10
3.1	Lancement	10
3.2	Synthèse	11

Correction orthographique : l'algorithme

Objectif de la section

Les élèves examinent un exemple concret d'algorithme piloté par les données : évaluer en quoi les comportements programmés d'un correcteur orthographique ressemblent à ceux des humains, et en quoi ils en diffèrent.

Lancement

Activité

Considérez cette phrase avec votre partenaire :

« Le chein aboit si for que tout le vossinage écoute. »

- Que pensez-vous que l'auteur·e voulait écrire ?
- Préparez-vous à décrire *comment* vous avez corrigé l'orthographe :
 - Comment repérez-vous les mots mal orthographiés ?
 - Comment corrigez-vous les mots mal orthographiés ?
 - Comment décririez-vous chaque type d'erreur commise par l'auteur·e ?

Réflexion

- **Comment avez-vous trouvé les mots mal orthographiés ?**
 - Les réponses varieront, mais la plupart des gens diront « je l'ai su dès que je l'ai vu ». C'est important à souligner !
- **Quand vous trouviez un mot mal orthographié, comment avez-vous trouvé la correction ?**
 - Réponses possibles : « Il n'y avait qu'un seul mot possible ! » / « Je pouvais le deviner d'après le reste de la phrase. » / « J'ai fait cette même erreur de frappe un million de fois. »
- **L'auteur·e a fait quatre types d'erreurs différents dans cette phrase. Lesquels ?**
 - **chein** Échange des lettres e et i (swap)
 - **aboit** Frappe t au lieu de e (substitution)
 - **for** Oubli de la lettre t à la fin (suppression)
 - **vossinage** Ajout d'un s en trop (insertion)

Point clé

Quand les humains corrigent l'orthographe, ils s'appuient sur **le contexte, l'expérience et leurs connaissances**. Le processus de correction orthographique varie non seulement d'une personne à l'autre, mais aussi selon la situation. Face à un mot technique complexe mal orthographié, par exemple, vous utiliseriez probablement une stratégie de correction différente.

Exploration

Le correcteur orthographique intégré à votre navigateur web est un exemple classique d'algorithme piloté par les données. Pour mieux comprendre en quoi les machines fonctionnent différemment de nous, nous allons suivre l'algorithme du programme SPELL original.

Activité

- Pour compléter **Le Premier Correcteur Orthographique**, vous serez l'ordinateur, en suivant l'algorithme d'un des premiers correcteurs orthographiques.
- Réfléchissez à l'activité et comparez-la à vos propres stratégies.

Notes pour l'enseignant·e

Après que les élèves ont terminé l'activité, invitez-les à partager leurs réponses aux trois questions de réflexion ci-dessous en petits groupes.

Réflexion

- **Quelles sont, selon vous, les limites de l’algorithme du premier correcteur orthographique ?**
 - Le nombre de façons de mal orthographier un mot que l’algorithme peut détecter est extrêmement limité. Par exemple, si deux lettres distinctes dans un mot sont incorrectes, le correcteur n’a *aucune chance* de proposer le bon mot !
- **En quoi l’algorithme du premier correcteur est-il similaire à la stratégie que VOUS utilisez pour vérifier l’orthographe ?**
 - Voyez ce que vos élèves proposent et discutez-en.
- **En quoi l’algorithme du premier correcteur est-il différent de la stratégie que VOUS utilisez ?**
 - Quand vous vérifiez l’orthographe, vous vous appuyez beaucoup sur le contexte, l’instinct et les connaissances acquises.
 - Le premier correcteur orthographique ne possédait — et n’utilisait — aucune de ces capacités.
 - En revanche, un humain n’aurait ni le temps, ni la capacité, ni la concentration pour énumérer systématiquement des milliers de mots potentiels, alors que l’ordinateur le fait sans difficulté.
 - Ainsi, les stratégies utilisées par l’humain et la machine peuvent être *extrêmement différentes* pour une tâche similaire.

Synthèse

- **En quoi les correcteurs orthographiques réussissent-ils à imiter les humains ?**
 - Ils ne le font pas ! Mais les résultats peuvent être similaires, selon une grande variété de facteurs.
- **Serait-il une bonne idée pour un humain d’essayer d’imiter l’algorithme d’un correcteur orthographique ? Pourquoi ?**
 - Non, ce serait long, laborieux et extrêmement sujet aux erreurs pour un humain.

Point clé

Le programme que nous venons de discuter repose sur la **force brute informatique** : les correcteurs primitifs modifient et transforment de façon répétée un mot mal orthographié, générant des dizaines de mots différents à vérifier dans le dictionnaire. L’algorithme est **piloté par les données** (*data-driven*) : sa qualité dépend *directement* de la représentativité du dictionnaire utilisé.

Correction orthographique en Pyret

Objectif de la section

Les élèves explorent à la fois l'algorithme et les jeux de données qui alimentent un correcteur orthographique écrit en Pyret, en découvrant que les algorithmes pilotés par les données sont au cœur de l'IA.

Lancement

Activité

- Examinons un programme de correction orthographique écrit en Pyret.
- Ouvrez le **fichier de démarrage du correcteur orthographique** (`Correcteur-Orthographique-Starter.arr`), enregistrez une copie et cliquez sur « Run ».
- Dans Pyret, un *commentaire* est toute ligne de code qui commence par `#`. Pyret ignore complètement les commentaires, les programmeurs les utilisent donc pour écrire des notes ! **Lisez les commentaires dans la zone de définitions.** Que vous apprennent ces notes ?
- Complétez la première section de **Un correcteur orthographique en Pyret**.

Dictionnaires disponibles Le fichier de démarrage est **autonome** : toutes les fonctions sont implémentées directement dans le code, sans dépendance externe. Il contient un dictionnaire embarqué :

Nom	Taille	Description
WORDS-XS-FR	100 mots	Dictionnaire minimal (mots de 5 lettres), embarqué
WORDS-S-FR	5 000 mots	À charger depuis <code>dictionaries/WORDS-S-FR.txt</code>
WORDS-M-FR	13 000 mots	À charger depuis <code>dictionaries/WORDS-M-FR.txt</code>
WORDS-L-FR	40 000 mots	À charger depuis <code>dictionaries/WORDS-L-FR.txt</code>

L'alphabet utilisé (`ALPHABET-FR-SIMPLE`) inclut les 26 lettres de base + les 6 accents les plus courants (é, è, ê, à, ù, ç). Une version complète (`ALPHABET-FR`) est également disponible avec æ, œ, ë, ï, ô, ö, û, ü.

Particularités du français

Le français pose un défi supplémentaire par rapport à l'anglais pour la correction orthographique : les **accents** (é, è, ê, ë, à, â, ç, ù, û, ü, ô, î, ï, etc.). Une erreur fréquente en français est l'oubli ou la confusion d'accent (« écoute » au lieu de « écoute », « eleve » au lieu de « élève »). L'algorithme de distance d'édition traite chaque caractère accentué comme un

caractère distinct : remplacer e par é compte pour une édition (substitution), au même titre que remplacer k par l.

Code Pyret

Fonctions disponibles dans le fichier de démarrage :

<code>subs(mot, alpha)</code>	Remplace chaque lettre une par une par toutes les lettres de l'alphabet. Ex. : <code>subs("chein", ALPHABET-FR-SIMPLE)</code> génère « ahien, bhien, ..., chien, ... »
<code>swaps(mot)</code>	Échange chaque paire de lettres adjacentes. Ex. : <code>swaps("chein")</code> génère « hcein, cehin, chien, chein »
<code>insertions(mot, alpha)</code>	Insère chaque lettre de l'alphabet à chaque position. Ex. : <code>insertions("aboi", ALPHABET-FR-SIMPLE)</code> génère « aaboi, baboi, ..., aboie, ... »
<code>deletions(mot)</code>	Supprime chaque lettre une par une. Ex. : <code>deletions("vossinage")</code> génère « ossinage, ..., voisinage, ... »
<code>only-real(liste, dico)</code>	Filtre une liste pour ne garder que les mots présents dans le dictionnaire.
<code>alt-words(mot, dico, n, alpha)</code>	Combine toutes les fonctions et trouve les mots du dictionnaire à distance $\leq n$.

Exploration

Activité

- **Comment le travail du correcteur s'est-il comparé à vos efforts pour être un ordinateur dans « Le Premier Correcteur Orthographique » ?**
 - Il a trouvé tout ce que nous avons trouvé, et *plus d'une centaine d'autres* insertions et substitutions en quelques secondes !
- **Toutes les suggestions de Pyret étaient-elles utiles ?**
 - Non ! La plupart des mots proposés étaient du charabia.

Point clé**Distance d'édition**

Tous les « mots » que notre correcteur a générés dans la première section impliquaient un *seul* changement. Le terme technique pour cela est la **distance d'édition** (*edit distance*) : la distance d'édition pour tous ces mots était de 1, ce qui signifie qu'ils étaient à une seule permutation, substitution, insertion ou suppression du mot original.

La distance d'édition est une façon de **mesurer la similarité de deux chaînes de caractères** !

Activité

- Voyons comment Pyret génère des listes de mots avec une distance d'édition plus grande et affine ses suggestions pour supprimer les mots absurdes.
- Complétez le reste de **Un correcteur orthographique en Pyret**.

Réflexion

- **Qu'avez-vous découvert sur les dictionnaires dans le fichier de démarrage ?**
- **Pourquoi `alt-words` a-t-il renvoyé un nombre différent de résultats selon le dictionnaire utilisé ?**
 - Parce que les petits dictionnaires ne connaissaient pas autant de mots, donc `only-real` supprimait des mots qui existent pourtant dans les plus grands dictionnaires.

Point clé

Quand on fournit des données *plus représentatives* à notre correcteur orthographique, on obtient de meilleurs résultats *sans changer le code* du correcteur.

L'efficacité des algorithmes pilotés par les données est **directement liée** à la **représentativité** des données.

Le même algorithme qui semble presque « intelligent » avec 10 000 des mots les plus utilisés en français peut sembler *inutile* avec un jeu de données de seulement 100 mots, ou avec les noms de 10 000 composés chimiques.

Synthèse

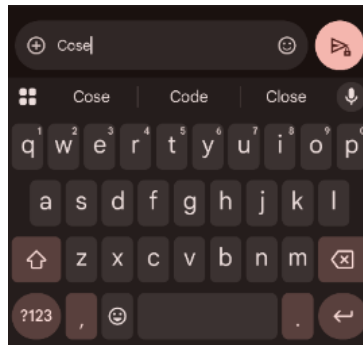


FIGURE 1 – Capture d’écran d’une application de messagerie proposant des alternatives pour un mot possiblement mal orthographié (l’original propose « Cose » → « Code », « Close »).

Réflexion

- **Quel type d’*algorithme* pensez-vous que l’application a utilisé pour trouver des alternatives possibles ?**
 - Les réponses varieront.
 - Les élèves peuvent faire référence aux algorithmes discutés dans la première moitié de la leçon.
 - Ils peuvent aussi imaginer des algorithmes plus complexes – par exemple, des algorithmes qui prennent en compte la proximité des lettres sur le clavier !
- **Quel type de *données* pensez-vous que l’application de correction a utilisées ?**
 - Quels mots l’utilisateur est le plus susceptible de taper.
 - Le sujet de la conversation et le mot suivant le plus probable.
 - Le dictionnaire de mots dans lequel l’application pioche.

Notes pour l’enseignant·e

Les élèves discuteront d’une capture d’écran similaire d’une application de messagerie dans la leçon sur les Modèles Génératifs. Dans cette leçon-là, cependant, ils exploreront comment l’IA générative utilise des algorithmes pilotés par les données pour déterminer le prochain mot à produire.

Définir des valeurs

Objectif de la section

Les élèves découvrent la définition de variables en Pyret, motivés par le besoin de sauvegarder des opérations coûteuses en calcul pour pouvoir réutiliser leurs résultats.

Lancement

Activité

Expérimentons avec une liste de suggestions plus grande...

- Dans la fenêtre d'interactions à droite, évaluez `alt-words("plation", WORDS-L-FR, 4, ALPHABET-FR-SIMPLE)`.
 - Combien de temps Pyret a-t-il mis pour générer toutes les substitutions, suppressions, insertions et échanges, puis filtrer pour ne garder que les vrais mots?
 - *Un bon moment !*
 - Est-ce que lui demander de refaire *la même chose* irait plus vite ? Pourquoi ?
 - Essayez ! Que s'est-il passé ? Pouvez-vous expliquer pourquoi ?
 - *Cela a encore pris une éternité, parce qu'on lui demande de refaire le même travail.*

Quand on demande à Pyret de faire quelque chose encore et encore, il n'a aucun moyen de savoir qu'il devrait sauvegarder son travail ! Résultat : *il refait le travail encore et encore !* Heureusement, le monde des mathématiques a une solution : **définir une variable**.

Code Pyret

Évaluez `resultats = alt-words("plation", WORDS-L-FR, 4, ALPHABET-FR-SIMPLE)`.

- Est-ce que quelque chose est revenu ? Pourquoi ?
- **Non !** Définir une variable ne produit rien, **exactement comme en maths !**
- $x = 4$ n'a pas de « réponse » parce que les définitions ne sont pas des expressions.
- Maintenant, évaluez `resultats` trois fois. Que se passe-t-il ?
- *C'était ultra-rapide !*
- Cliquez sur « Run » et évaluez `resultats`. Que se passe-t-il ?
- *On obtient un message d'erreur ! Pyret a « oublié » notre variable.*
- Évaluez `sentiment = "Joie"` et appuyez sur Entrée. Que va-t-il se passer si on évalue `sentiment` ?
- *On obtiendra "Joie".*

Quand on veut que Pyret *se souvienne* d'une définition, on l'ajoute à la **zone de définitions**. Chaque fois qu'on clique sur « Run », Pyret ré-exécute tout le code, y compris la définition !

Activité

- Ajoutez `resultats = alt-words("plation", WORDS-L-FR, 4, ALPHABET-FR-SIMPLE)` comme **dernière ligne de la zone de définitions**, et cliquez sur « Run ». Évaluez `resultats` dans la zone d'interactions.
- Que se passe-t-il si vous ajoutez une autre définition qui *a aussi* le nom `resultats` ?

Synthèse

Réflexion

- **Pourquoi les définitions de variables sont-elles précieuses ?**
 - On peut sauvegarder un travail coûteux en calcul.
- **Quels types de valeurs peut-on définir, au-delà des tables de corrections orthographiques ?**
 - N'importe quoi !
- **Pourquoi ne peut-on pas définir deux variables avec le même nom ?**
 - Pyret n'autorise un nom de variable qu'à avoir une seule valeur.
- **Maintenant que nous pouvons générer et sauvegarder des suggestions orthographiques, que pourrions-nous vouloir *en faire* ?**
 - Trier les résultats, en mettant les suggestions avec la plus courte distance d'édition en premier.
 - Afficher seulement le « top 5 » plutôt qu'une table avec des centaines d'entrées.
 - Peut-être faire un graphique ?

Annexe A : Dictionnaires français

Les dictionnaires français suivants sont fournis dans le dossier `dictionaries/` :

Fichier	Mots	Usage
<code>WORDS-XS-FR.txt</code>	169 mots de 5 lettres	Exercices de découverte
<code>WORDS-S-FR.txt</code>	5 000 mots courants	Travail en classe
<code>WORDS-M-FR.txt</code>	13 000 mots courants	Exploration
<code>WORDS-L-FR.txt</code>	40 000 mots	Projets

Ces dictionnaires ont été extraits du dictionnaire Hunspell français (`fr_FR`) et filtrés pour ne contenir que des mots en minuscules avec accents. Le dictionnaire `WORDS-XS-FR` a été sélectionné manuellement pour garantir des mots courants et reconnaissables par les élèves.

Annexe B : Le français et la distance d'édition

Le français pose des défis spécifiques pour la correction orthographique :

1. **Accents** : 14 caractères accentués (à, â, é, è, ê, ë, î, ï, ô, ö, ù, û, ü, ç, æ, œ). Chaque accent manquant ou erroné compte comme une édition.
2. **Lettres muettes** : De nombreux mots français ont des lettres finales non prononcées (*parlent*, *chanter*, *prix*), sources d'erreurs fréquentes.
3. **Homophones** : a/à, ou/où, et/est, son/sont, on/ont, etc. La distance d'édition ne capture pas les erreurs grammaticales ; une correction contextuelle est nécessaire.
4. **Liaisons et élisions** : l'avion, d'abord, qu'il, etc. rendent la tokenisation plus complexe.

Ces défis expliquent pourquoi les correcteurs orthographiques modernes combinent distance d'édition *et* modèles de langue entraînés sur de vastes corpus.

Annexe C : Fichiers du projet

Fichier	Description
<code>Correcteur-Orthographique-Starter.arr</code>	Fichier Pyret autonome (tout réimplémenté)
<code>algorithmes-pilotés-donnees.tex</code>	Document de cours (ce fichier)
<code>dictionaries/WORDS-XS-FR.txt</code>	100 mots français de 5 lettres
<code>dictionaries/WORDS-S-FR.txt</code>	5 000 mots français courants
<code>dictionaries/WORDS-M-FR.txt</code>	13 000 mots français
<code>dictionaries/WORDS-L-FR.txt</code>	40 000 mots français
<code>images/</code>	Illustrations de la leçon

À noter : contrairement à la version anglaise originale qui charge une bibliothèque distante, le fichier `.arr` est entièrement autonome. Toutes les fonctions (`subs`, `swaps`, `insertions`, `deletions`, `only-real`, `alt-words`) sont réimplémentées en Pyret pur. L'alphabet inclut les caractères accentués français, et le dictionnaire WORDS-XS-FR (100 mots) est embarqué pour des démonstrations instantanées. Les grands dictionnaires sont à charger depuis les fichiers `.txt` du dossier `dictionaries/`.

À propos de ces supports

Ces supports ont été développés en partie grâce au soutien de la *National Science Foundation* (bourses 1042210, 1535276, 1648684, 1738598, 2031479 et 1501927).

Bootstrap par la Communauté Bootstrap est sous licence Creative Commons 4.0 Unported. Cette licence n'accorde pas la permission d'organiser des formations ou du développement professionnel.

Traduction et adaptation française réalisées en juillet 2026.

Source originale : <https://www.bootstrapworld.org/materials/fall2026/en-us/lessons/data-driven-algorithms/>